

Problem Description: Ram has provide inputs two numbers 'p' and 'q' to Sakthi. He wants to creates a matrix of size $p \times q$ (p rows and q columns) in which every elements is either Y or O. The Ys and Os must be filled alternatively, the matrix should have outermost rectangle of Ys, then a rectangle of Os, then a rectangle of Ys, and so on..

Constraints:

$$1 \leq p, q \leq 1000$$

Input Format:

Input lines must be how many rows and columns in that matrix, also values must be separate space.

Output Format:

Print the output in a separate lines.

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

6 7

Test Case 2

INPUT (STDIN)

5 8

EXPECTED OUTPUT

```
Y Y Y Y Y Y Y
Y 0 0 0 0 0 Y
Y 0 Y Y Y 0 Y
Y 0 Y Y Y 0 Y
Y 0 0 0 0 0 Y
Y Y Y Y Y Y Y
```

EXPECTED OUTPUT

```
Y Y Y Y Y Y Y Y
Y 0 0 0 0 0 0 Y
Y 0 Y Y Y Y 0 Y
Y 0 0 0 0 0 0 Y
Y Y Y Y Y Y Y Y
```

✓ Mandatory Test Cases

Test Case 1

KEYWORD

```
while(top<=bottom &&
right>=left)
```

✓ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

12

Test Case 2

TOKEN COUNT

290

Test Case 3

NLOC

52

Code

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int p, q;
6      int top, bottom, left, right;
7      int i, j;
8      int value = 1;
9
10     scanf("%d %d", &p, &q);
11
12     top = 0;
13     bottom = p - 1;
14     left = 0;
15     right = q - 1;
16
17     char matrix[p][q];
18
19     while (top <= bottom && right >= left)
20     {
21         /* Top row */
22         for (j = left; j <= right; j++)
23         {
24             if (value == 1)
25                 matrix[top][j] = 'Y';
26             else
27                 matrix[top][j] = '0';
28         }
29
30         /* Right column */
31         for (i = top + 1; i <= bottom; i++)
32         {
33             if (value == 1)
34                 matrix[i][right] = 'Y';
```

```
35         else
36             matrix[i][right] = '0';
37     }
38
39     /* Bottom row */
40     if (bottom > top)
41     {
42         for (j = right - 1; j >= left; j--)
43         {
44             if (value == 1)
45                 matrix[bottom][j] = 'Y';
46             else
47                 matrix[bottom][j] = '0';
48         }
49     }
50
51     /* Left column */
52     if (right > left)
53     {
54         for (i = bottom - 1; i > top; i--)
55         {
56             if (value == 1)
57                 matrix[i][left] = 'Y';
58             else
59                 matrix[i][left] = '0';
60         }
61     }
62
63     top++;
64     bottom--;
65     left++;
66     right--;
67
```

```
68         value = 1 - value;
69     }
70
71     /* Print matrix */
72     for (i = 0; i < p; i++)
73     {
74         for (j = 0; j < q; j++)
75         {
76             printf("%c", matrix[i][j]);
77
78             if (j < q - 1)
79                 printf(" ");
80         }
81
82         printf("\n");
83     }
84
85     return 0;
86 }
```

Output:

